LAB 4 | Adaptive optimizers for CNN

FASHION-MNIST CLOTHING CLASSIFICATION

Build one compact convolutional classifier and observe how five Keras optimizers behave on the same images, validation split, initial weights and number of epochs.

Learning objectives

- Prepare CNN input tensors with an explicit grayscale channel.
- Keep convolutional architecture fixed during optimizer comparison.
- Plot and interpret real validation-accuracy curves.

Reproducible experiment settings

- Fashion-MNIST | training 2400 | validation 500 | test 1000.
- CNN: Conv2D(16) → MaxPool → Conv2D(32) → MaxPool → Flatten → Dense(32) → Dense(10).
- Fixed EPOCHS=4, BATCH_SIZE=64, identical starting weights and image subsets.
- Five independent optimizer runs: SGD, Adam, RMSprop, Adadelta and Adagrad.

Run instructions

1. In Google Colab, create a notebook, paste the complete Python solution and run it.
2. For local Python, install TensorFlow, NumPy and Matplotlib: pip install tensorflow numpy matplotlib
3. The public dataset downloads on the first execution. Actual performance numbers are created only when the code is run.
4. The Python (.py) source file is provided separately for direct execution and easier copying.

Complete worked solution | line-commented Python code

Instructional comments accompany each executable statement. The standalone .py file contains exactly this code; output numbers and charts are generated by running it.

```python
# LAB 2: Compare SGD, Adam, RMSprop, Adadelta, and Adagrad in a CNN.
# Run in Google Colab or locally after: pip install tensorflow numpy matplotlib
# Dataset: Fashion-MNIST, a public 10-class clothing image dataset.

import os  # Control TensorFlow's informational messages.
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"  # Hide routine startup logs.
import csv  # Save measured outcomes to a portable CSV file.
import time  # Measure actual training time for each optimizer.
from pathlib import Path  # Save charts and tables in one folder.
import numpy as np  # Select deterministic dataset subsets.
import matplotlib.pyplot as plt  # Plot validation accuracy histories.
import tensorflow as tf  # Use Keras to implement a convolutional model.

SEED = 42  # Keep the dataset split and initialization repeatable.
TRAIN_N = 2400  # Use a smaller dataset for an affordable classroom lab.
VAL_N = 500  # Reserve these labels for validation during comparison.
TEST_N = 1000  # Hold out separate examples for testing.
EPOCHS = 4  # Use an identical number of training epochs in all runs.
BATCH_SIZE = 64  # Keep batch size constant when changing optimizers.
OUT = Path("lab2_cnn_optimizer_results")  # Set the output folder name.
OUT.mkdir(parents=True, exist_ok=True)  # Ensure the folder exists.
tf.keras.utils.set_random_seed(SEED)  # Seed Python, NumPy, and TensorFlow.

(x_all, y_all), (x_test_all, y_test_all) = tf.keras.datasets.fashion_mnist.load_data()  # Download Fashion-MNIST.
rng = np.random.default_rng(SEED)  # Prepare a reproducible random selector.
train_ids = rng.permutation(len(x_all))[:TRAIN_N + VAL_N]  # Select training and validation images.
test_ids = rng.permutation(len(x_test_all))[:TEST_N]  # Select disjoint official test images.
x_train = x_all[train_ids[:TRAIN_N]].astype("float32") / 255.0  # Scale training image pixels.
y_train = y_all[train_ids[:TRAIN_N]]  # Keep matching training labels.
x_val = x_all[train_ids[TRAIN_N:]].astype("float32") / 255.0  # Scale validation pixels.
y_val = y_all[train_ids[TRAIN_N:]]  # Keep matching validation labels.
x_test = x_test_all[test_ids].astype("float32") / 255.0  # Scale held-out test pixels.
y_test = y_test_all[test_ids]  # Keep matching test labels.
x_train = x_train[..., np.newaxis]  # Add the required grayscale CNN channel dimension.
x_val = x_val[..., np.newaxis]  # Give validation data shape (N,28,28,1).
x_test = x_test[..., np.newaxis]  # Give test data the same image shape.

def build_cnn():  # Build a compact image classifier for all five optimizers.
    return tf.keras.Sequential([  # Create a sequential convolutional network.
        tf.keras.layers.Input(shape=(28, 28, 1)),  # Expect 28-by-28 grayscale images.
        tf.keras.layers.Conv2D(16, 3, activation="relu"),  # Detect low-level visual patterns.
        tf.keras.layers.MaxPooling2D(),  # Shrink feature maps to reduce calculations.
        tf.keras.layers.Conv2D(32, 3, activation="relu"),  # Learn more complex image features.
        tf.keras.layers.MaxPooling2D(),  # Compress the second group of maps.
        tf.keras.layers.Flatten(),  # Convert feature maps into one vector.
        tf.keras.layers.Dense(32, activation="relu"),  # Combine extracted visual features.
        tf.keras.layers.Dense(10, activation="softmax"),  # Return probabilities for 10 clothes classes.
    ])  # Complete CNN architecture.

# Students can manually replace the one optimizer expression in model.compile.
# These explicit rates are teaching starting points, not universal defaults.
OPTIMIZERS = [  # Declare five independent optimizer configurations.
```

```
    ("SGD", lambda: tf.keras.optimizers.SGD(learning_rate=0.01)),  # Nonadaptive baseline.
    ("Adam", lambda: tf.keras.optimizers.Adam(learning_rate=0.001)),  # Moment estimates.
    ("RMSprop", lambda: tf.keras.optimizers.RMSprop(learning_rate=0.001)),  # Moving squared gradients.
    ("Adadelta", lambda: tf.keras.optimizers.Adadelta(learning_rate=1.0)),  # Windowed update scales.
    ("Adagrad", lambda: tf.keras.optimizers.Adagrad(learning_rate=0.01)),  # Accumulated squared gradients.
]  # Finish the comparison group.

tf.keras.utils.set_random_seed(SEED)  # Control initialization of the reference network.
reference_model = build_cnn()  # Create an untrained CNN with known starting weights.
initial_weights = reference_model.get_weights()  # Save an identical start for every optimizer.
del reference_model  # Discard this model without training it.
results = []  # Prepare rows for a measured results table.
histories = {}  # Prepare validation accuracy trajectories for plotting.
for opt_name, make_optimizer in OPTIMIZERS:  # Compare one optimizer at a time.
    tf.keras.utils.set_random_seed(SEED)  # Reapply seeds at the start of each run.
    model = build_cnn()  # Rebuild the same CNN architecture.
    model.set_weights(initial_weights)  # Reset the model to the same initial weights.
    optimizer = make_optimizer()  # Select the optimizer under comparison.
    model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy", metrics=["accuracy"])  # Configure categorical learning.
    start = time.perf_counter()  # Note start time directly before training.
    history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=EPOCHS, batch_size=BATCH_SIZE, shuffle=True, verbose=0)  # Train on fixed data.
    elapsed = time.perf_counter() - start  # Record training time on this machine.
    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)  # Assess held-out examples.
    best_val = max(history.history["val_accuracy"])  # Calculate the best validation epoch.
    histories[opt_name] = history.history["val_accuracy"]  # Store the epoch-by-epoch curve.
    results.append({"optimizer": opt_name, "learning_rate": float(optimizer.learning_rate.numpy()), "best_val_accuracy": round(best_val, 4), "test_accuracy": round(test_accuracy, 4),
"test_loss": round(test_loss, 4), "train_seconds": round(elapsed, 2)})  # Record observed scores.
    print(f"{opt_name:8} | val={best_val:.3f} | test={test_accuracy:.3f} | time={elapsed:.1f}s")  # Print true outcomes.

plt.figure(figsize=(8, 4.5))  # Create one validation accuracy comparison figure.
for name, accuracy_curve in histories.items():  # Plot each optimizer history.
    plt.plot(range(1, EPOCHS + 1), accuracy_curve, marker="o", label=name)  # Draw actual validation values.
plt.title("CNN on Fashion-MNIST: optimizer comparison")  # Label the comparison context.
plt.xlabel("Epoch")  # Show epochs horizontally.
plt.ylabel("Validation accuracy")  # Show held-out validation performance vertically.
plt.xticks(range(1, EPOCHS + 1))  # Display one tick per training epoch.
plt.grid(alpha=0.25)  # Add a light grid for reading trends.
plt.legend()  # Display the five optimizer names.
plt.tight_layout()  # Avoid clipped graph labels.
plt.savefig(OUT / "cnn_validation_accuracy.png", dpi=160)  # Export the observed curves.
plt.close()  # Free the matplotlib figure.
with (OUT / "cnn_optimizer_results.csv").open("w", newline="", encoding="utf-8") as file:  # Save the result summary.
    writer = csv.DictWriter(file, fieldnames=list(results[0].keys()))  # Set stable output headings.
    writer.writeheader()  # Export column names first.
    writer.writerows(results)  # Export the five measured experiments.
print(f"Results and plot saved to: {OUT.resolve()}")  # Tell students where files are located.
```

Expected generated files

- lab2_cnn_optimizer_results/cnn_optimizer_results.csv
- lab2_cnn_optimizer_results/cnn_validation_accuracy.png

How to interpret your results (sample answer framework)

Which optimizer improves fastest?

Does the fastest mean the best?

Why did we reset weights?

How to make a stronger study?

Practical cautions and further reading